52

BÖLÜM: DEĞIŞIKLIK TALEPLERI, HATALAR (BUGS) VE MEVCUT (BROWNFIELD) SISTEMLER | MODULE: CHANGE REQUESTS, BUGS, AND BROWNFIELD SYSTEMS

Mevcut Sistemleri Kurtarma (Brownfield): Kodu Değil, Niyeti Kurtarın!

Brownfield Recovery: Reverse-Engineering Intent, Not Transcribing Code

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Toprağın altından kırık bir antik vazo çıkaran bir arkeolog düşünün: Arkeolog vazonun üstündeki çamuru ve tozu kopyalayıp müzeye koymaz! Çamuru dikkatlice temizler ve altındaki paha biçilmez altın deseni (Niyeti) ortaya çıkarır. Brownfield recovery, 10 yıllık eski kodun üstündeki SOAP ve XML çamurlarını temizleyip saf iş mantığını kurtarmaktır!

🇬🇧 ENGLISH (ELI5)

Think of an archaeologist excavating an ancient vase: The archaeologist doesn't duplicate the dirt and mud crust into museum showcases! They delicately brush off the grime to reveal the exquisite gold inlay beneath (The Intent). Brownfield recovery washes away 10 years of legacy XML and SOAP grime to rescue pure business logic!

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Mevcut (Brownfield) Sistemlerde Tersine Mühendisliğin Felsefesi: Dünyadaki en kritik yazılımların %90'ı sıfırdan yazılmaz; yıllardır çalışan brownfield sistemlerdir. Martinelli'nin 'reverse-engineer' becerisi şu manifesto ile başlar: 'Siz kodu transkribe etmiyorsunuz. Siz o kodun tatmin etmek için inşa edildiği İŞ NİYETİNİ kurtarıyorsunuz ve bunu bir iş analistinin kod yazılmadan önce yazacağı seviyede kâğıda döküyorsunuz!' Concretely: 1) Controller ve View'lar use case değildir; use case'in kanıtlarıdır. 2) Koddaki her bir `if (x > y)` kontrolü muhtemel bir iş kuralı (BR-XXX) adayıdır. 3) Veritabanı şeması doğrudan Entity Model'e dönüştürülür. 4) Eğer bir use case adımı yalnızca o kodu okumuş birinin anlayacağı terimler içeriyorsa derhal silinmeli ve iş diline çevrilmelidir.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

The Philosophy of Brownfield Reverse-Engineering: Greenfield green-pasture projects are rare; enterprise software is overwhelmingly brownfield. The AIUP reverse-engineering discipline is guided by an uncompromising stance: 'You are not transcribing the code. You are recovering the intent the code was built to satisfy, and writing it down at the level a business analyst would have written it before implementation.' Concretely: 1) Endpoints, controllers, and screens are not use cases; they are circumstantial evidence of use cases. 2) Guard clauses and validation branches are candidate business rules (BR-XXX), requiring translation into domain vernacular. 3) Database tables are evidence of entity model entities. 4) If a recovered step only makes sense to an engineer who has examined the repository source, it violates Zone 3 and must be rewritten into observable stakeholder behavior.

⚡ Modern Java 25 & Clean Architecture Uygulaması

⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Legacy Spagettiden Kurtarılan Saf İş Niyeti
package com.maang.aiup.brownfield;

public class LegacyRecoveryDemo {
    // ❌ ESKİ MİRAS KOD (10 Yıllık Spagetti):
    // public void proc_99(int t, double a) { if (t == 4 && a > 500) ... }

    // ✅ KURTARILAN İŞ MODELİ (Clean & Expressive):
    public record CorporateOrder(String companyId, double amount, boolean hasDiscount) {}

    public static class RecoveredCorporateOrderPolicy {
        public static boolean isEligibleForVipProcessing(CorporateOrder order) {
            // Koddaki 't == 4' ifadesinin kurumsal müşteri olduğunu kurtardık!
            return order.amount() > 500.0 && order.hasDiscount();
        }
    }

    void main() {
        var order = new CorporateOrder("CORP-101", 750.0, true);
        System.out.println("✅ Miras Koddan Kurtarılan İş Kuralı Sonucu: "
            + RecoveredCorporateOrderPolicy.isEligibleForVipProcessing(order));
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

Eski sistemi kurtarırken 'hazır görmüşken bu hatayı da şartnamede düzelteyim' demeyin! Kurtarma (Discovery) ile Karar Verme (Deciding) iki ayrı aşamadır. Önce ne olduğunu yazın, sonra incelemede düzeltin.

⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Do not attempt to fix business bugs while reverse-engineering legacy code. Recovery is discovery; decision-making is a subsequent stakeholder activity. Document what exists first.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

- Miras sistemlerde kod transkribe edilmez; kodun çözdüğü iş niyeti kurtarılır.
- Koddaki if-else guard blokları şartnamede BR-XXX iş kurallarına dönüştürülür.
- Kurtarma (keşif) ile karar verme (düzeltme) süreçleri birbirinden kesin olarak ayrılmalıdır.

🎯 Key Architectural Takeaways

- In brownfield systems, do not transcribe code; recover the underlying business intent.
- Guard clauses and validations in legacy code translate into candidate BR-XXX business rules.
- Discovery must remain separate from decision-making during legacy specification recovery.